

Darrel Feiro Wihandi

✉ dfwihand@uwaterloo.ca — 🌐 Portfolio — 📄 Darrel Wihandi — 📄 DarrelFW321

Education

University of Waterloo

BSE Software Engineering Candidate

Relevant Coursework: Data Structures & Data Management, Digital Computers, Database Management, Software Engineering Principles, Combinatorics, Compilers

Skills

Languages: C++, Python, Rust, SQL, JavaScript/TypeScript

Systems & Tools: LLVM, CMake, Linux, Git, Docker, Kubernetes, AWS, gRPC, PostgreSQL, Redis, OpenTelemetry, WebAssembly

ML/Backend: FastAPI, PyTorch, LangGraph/LangChain, SQLite, Next.js

Experience

Software Engineering Intern | C++, Python, gRPC, PostgreSQL, Redis

Sept. 2026 – Present

Carta

- Built a **self-service enterprise SSO workflow** with C++ gRPC APIs for **configuration CRUD operations**, replacing staff-assisted setup and reducing estimated onboarding time from **1–2 hours to within 15 minutes**.
- Extended the integration with **SCIM provisioning workflows** for user creation, deactivation, and group membership changes, synchronously propagating identity and access updates to downstream internal systems.
- Built a **Python MockIdP** to validate **SSO and SCIM provisioning workflows**, provisioning test identity providers and simulating authentication responses for automated and manual testing of **SSO enforcement and portfolio transfers**.

Graphics Compiler Engineer 📡 | C++, Compiler Internals, Testing

Feb. 2025 – Present

NVIDIA Corporation – Open Source Contributor

- Implementing Slang texture intrinsics across **SPIR-V, HLSL, WGSL, and CUDA** code generation backends.
- Designed cross-backend validation tests to verify consistent behavior and prevent language-feature regressions.

Machine Learning Systems Engineer | Python, LangGraph, OpenTelemetry, SQLite

May 2026 – Present

Wat.AI — TRACE

- Built **TRACE**, an AI-agent reliability framework that pinpoints failures in multi-step workflows and generates reproducible test cases, reducing investigation time by **40%**.
- Engineered **Python trace analysis and deterministic state validation** to distinguish root causes from downstream errors and produce evidence-backed failure reports.
- Automated **500+ evaluation runs** across **30+ adversarial scenarios**, integrating regression replay into CI to catch recurring failures and verify fixes preserve valid agent behavior.

ML/Gen AI Software Engineer | Python, Azure OpenAI, LangChain, FastAPI

Jan. 2026 – May 2026

Rogers Communications

- Built an enterprise metadata automation platform with pipeline orchestration, confidence scoring, and structured validation to route low-confidence outputs for human review.
- Reduced metadata evaluation time from **6 min to 2 min per asset**, projecting **67 hours saved per 1,000 records** across large-scale catalog operations.

Software Engineer & DevOps Intern | Python, AWS, Kubernetes

May 2025 – Aug. 2025

Eon Media

- Engineered GPU-accelerated video inference pipelines on AWS EKS for real-time object tracking, OCR, and frame-level analytics over sports footage.
- Automated deployment and infrastructure workflows across EKS, EC2, and S3, reducing deployment overhead by **50%**.

Projects

Flux 📡 | C++17, LLVM, WebAssembly, React, FastAPI

Feb. 2026 – Present

DSL Compiler with Live Pipeline Visualizer

- Designed and implemented a statically typed DSL with a lexer, recursive descent parser, AST, type checker, custom IR, and LLVM-backed native code generation.
- Built optimization and benchmarking infrastructure to compare generated numerical kernels against C and NumPy, validating correctness and runtime behavior across compilation stages.

Dex 📡 | TypeScript, React Native, Swift, Three.js

Sept. 2026

AI Agent for 3D Room Design and AR Visualization — Hack the North

- Built an **AI agent that designs 3D spaces and visualizes them in AR**, translating voice commands into edits on LiDAR-scanned rooms with **on-device collision, clearance, and door-swing validation**.
- Engineered a **hybrid 3D asset pipeline** with parametric templates, graph-based assembly validation, and catalog reuse to bypass fresh generation; integrated **Meshy** for unmatched requests with previews before texturing.